嵌入式软件代码缺陷与安全漏洞检测系统

（ecdet V1.0）

用户操作手册

软件版本号：V1.0

文档版本号：V1.0

著作权人：冯雯雯

编制日期：2026 年 10 月 20 日

目　　录

1 系统概述 **5**

1.1 编制目的 5

1.2 系统定位与适用场景 5

1.3 设计思想 5

1.4 五层架构 6

1.5 主要技术指标 6

2 运行环境 **8**

2.1 硬件环境 8

2.2 软件环境 8

2.3 可选外部工具 8

3 安装与启动 **10**

3.1 安装步骤 10

3.2 命令行入口 10

3.3 启动 Web 控制台 10

3.4 容器化部署 10

3.5 升级、备份与卸载 11

4 命令行功能详解 **12**

4.1 scan 全量扫描 12

4.2 init-ccdb 构建复现 12

4.3 rules 规则集查询 12

4.4 report 报告重建 12

4.5 triage 告警分诊 13

4.6 fix 修复建议 13

4.7 impact 影响集分析 13

4.8 orchestrate 多智能体编排 13

4.9 serve 控制台服务 13

5 Web 控制台操作 **14**

5.1 界面结构 14

5.2 总览 15
5.3 工程管理 15
5.4 扫描运行 16
5.5 代码浏览 17
5.6 版本对比 18
5.7 告警中心 19
5.8 修复建议 20
5.9 大模型设置 21
5.10 界面通用交互 22
5.11 状态与结论的含义 22

6 检测能力说明 24

6.1 内置规则集 24
6.2 外部引擎接入 24
6.3 可证明性分级 24
6.4 合规标准映射 25
6.5 动态分析通道 25
6.6 多智能体协同 25
6.7 误报抑制机制 25

7 输出产物 27

7.1 SARIF 结果文件 27
7.2 HTML 检测报告 27
7.3 补丁与修复记录 27
7.4 基线文件 27

8 典型使用流程 28

8.1 首次全量扫描 28
8.2 大模型分诊 28
8.3 一键修复与验收 28
8.4 增量扫描与版本对比 28

9 常见问题与使用限制 29

9.1 常见问题 29

9.2 使用限制 29

10 数据与安全管理 30

10.1 部署边界 30

10.2 凭据管理 30

10.3 运行数据留存 30

10.4 路径安全 30

10.5 可追溯性 30

11 附录 31

附录 A 内置规则清单 31

附录 B 检测结果文件示例 32

附录 C 控制台服务接口一览 33

附录 D 输出目录结构 34

附录 E 术语表 35

1 系统概述

1.1 编制目的

本手册是嵌入式软件代码缺陷与安全漏洞检测系统（以下简称本系统或 ecdet）的配套用户文档，用于说明系统的功能构成、运行环境、安装部署方法、命令行与图形界面的操作步骤，以及检测结果的输出格式与含义。软件使用人员、集成测试人员和工程实施人员可依据本手册完成从工程注册、扫描执行到告警处置与修复验收的完整流程。

1.2 系统定位与适用场景

本系统面向汽车电子与工业控制领域的嵌入式 C/C++ 代码，提供代码缺陷与安全漏洞的自动化检测、告警研判与修复建议生成能力。系统以静态分析与动态执行双通道为骨架，以大模型多智能体协同为中枢，以工具验证与证据链为约束，兼顾检测精度与结果可解释性。

典型适用场景包括：

- 车载控制器、仪表、网关等 MCU 固件在代码提交前的缺陷自查；
- 工业控制设备固件在版本迭代过程中的增量缺陷比对；
- MISRA C、CERT C 等编码规范符合性检查与合规证据留存；
- 存量代码在安全整改阶段的漏洞定位、修复建议生成与修复后验收。

1.3 设计思想

嵌入式软件的缺陷往往同时具有“语言级”与“硬件语义级”两重成因。仅依靠通用静态分析工具，难以理解中断服务程序与任务之间的并发关系、寄存器读写属性约束等硬件语义；仅依靠编译告警，又无法覆盖越界、竞态与未定义行为。为此本系统采用三项核心设计：

- 静态与动态双通道：静态通道以规则引擎、抽象解释与有界模型检验覆盖可判定的缺陷形态；动态通道通过激励建模与 harness 自动生成，驱动被测函数在宿主环境真实执行，以崩溃退出码、未定义行为检测器输出与看门狗超时作为判定依据，形成可复现的概念验证证据。
- 大模型多智能体协同：将告警研判、修复建议生成等语义推理任务分解给规划、研判、动态测试、监控、修复、报告六类智能体，通过结构化交接逐级推进；大模型不承担最终裁决权，其结论必须通过证据核对与工具验证方可采纳。

- 工具验证与证据链约束：所有告警携带规则编号、代码位置、证据行与可证明性等级；所有修复补丁必须通过静态重扫与编译器语法检查才被采纳。

1.4 五层架构

系统按数据处理流程划分为五个层次，各层职责与实现模块如下表所示。

层次	名称	主要职责	实现模块
L1	编译环境复现	自动识别 Keil、IAR 等工程文件，重建编译数据库，产出 compile_commands.json	ecdet.l1_build
L2	统一表示层	将多引擎输出统一为 SARIF 2.1.0 单一数据模型，注入硬件语义上下文	ecdet.l2_ir
L3	双通道分析	静态：规则引擎、抽象解释、有界模型检验；动态：激励建模、harness 闭环、仿真执行	ecdet.l3_analysis
L4	多智能体协同	告警分诊、知识检索增强、动态测试、运行时监控、修复闭环、报告生成	ecdet.l4_llm
L5	聚合与输出	去重聚合、CWE/MISRA/CERT/GJB 映射、可证明性分级、HTML 报告与合规记录	ecdet.l5_report

除上述五层外，系统提供 Web 控制台（ecdet.webui）作为图形化操作入口，以纯标准库实现服务端与前端资源，离线环境可直接运行。

1.5 主要技术指标

项目	指标
内置检测规则	30 条（MISRA C:2012 子集 17 条、CERT C 子集 5 条、缺陷规则 4 条、硬件语义规则 4 条）
外部引擎	cppcheck、clang-tidy、ESBMC，检测到安装即自动接入
结果交换格式	SARIF 2.1.0
可证明性分级	Proven（已证明）、Detected（已检出）、Suspected（疑似）
合规映射	CWE、MISRA C:2012、CERT C、GJB 8114-2013

项目	指标
动态执行载体	宿主原生执行（gcc）、Unicorn CPU 级仿真、Renode 外设级仿真
核心依赖	Python 3.11 及以上，核心功能零第三方依赖

2 运行环境

2.1 硬件环境

环境	最低配置	推荐配置
开发环境	x86_64 处理器，4 核，8 GB 内存，20 GB 可用磁盘	x86_64 处理器，8 核及以上，16 GB 内存，100 GB SSD
运行环境	x86_64 处理器，2 核，4 GB 内存，10 GB 可用磁盘	x86_64 处理器，4 核及以上，8 GB 内存，50 GB SSD
显示环境	分辨率 1280×800	分辨率 1920×1080 及以上

2.2 软件环境

项目	要求
操作系统	Windows 10/11（64 位）、Linux（glibc 2.28 及以上）、macOS 12 及以上
运行时	Python 3.11 及以上（3.12 推荐）
包管理	uv（推荐）或 pip
浏览器	Chrome 100+、Edge 100+、Firefox 100+（仅 Web 控制台需要）
可选编译器	gcc 9 及以上（用于语法检查与动态执行通道）
可选外部引擎	cppcheck 2.10+、clang-tidy 14+、ESBMC 7+
可选大模型服务	任意 OpenAI 兼容端点（vLLM、Ollama、LM Studio 等）

系统核心功能采用 Python 标准库实现，未引入第三方运行时依赖，可在离线构建机上直接安装使用；外部引擎与大模型服务均为可选组件，未安装时系统自动降级并记录降级原因。

2.3 可选外部工具

下列工具由系统在运行时自动探测，探测到即启用，未探测到则跳过对应能力，不影响主流程执行。

工具	用途	缺失时的影响
gcc	补丁语法检查、动态通道 harness 编译	跳过语法检查，动态通道不可用
cppcheck	通用缺陷基线检测	减少一类告警来源
clang-tidy	AST 级规则检查	减少一类告警来源

工具	用途	缺失时的影响
ESBMC	有界模型检验	跳过界内验证，Proven 级结论减少

3 安装与启动

3.1 安装步骤

在项目根目录执行下列命令完成环境创建与安装。使用 uv 可自动创建虚拟环境并安装声明依赖。

```
# 安装（创建虚拟环境并同步依赖）
uv sync


# 验证安装
uv run ecdet --version
```

安装完成后，命令行入口 ecdet 即可使用。系统不写入系统级目录，所有运行数据默认写入当前工作目录下的输出子目录，卸载时直接删除项目目录即可。

3.2 命令行入口

系统提供九个子命令，覆盖从构建复现到报告生成的完整链路。执行 ecdet -h 可查看全部子命令。

```
usage: ecdet [-h] [--version]
             {scan,init-ccdb,rules,report,triage,fix,impact,orchestrate,serve} ...
```

3.3 启动 Web 控制台

执行下列命令启动图形化控制台，默认监听本机 8600 端口：

```
uv run ecdet serve --port 8600
# 浏览器访问 http://127.0.0.1:8600
```

首次启动时系统自动在工作目录下创建 webui_data 目录，并在其中初始化 SQLite 数据库文件 ecdet_web.db，用于保存工程信息、扫描运行记录、告警复核状态、大模型端点的连接配置以及每次运行对应的源码快照。

3.4 容器化部署

系统提供 Dockerfile，可将检测能力封装为镜像，便于在持续集成环境中调用。镜像内置 cppcheck 基线引擎，大模型服务通过环境变量指向宿主机。

```
docker build -t ecdet:latest .
docker run --rm -v /path/to/firmware:/work ecdet:latest scan /work
```

3.5 升级、备份与卸载

升级时在项目根目录重新执行依赖同步命令即可，系统不对运行数据目录做结构变更时的强制迁移，历史运行记录与源码快照保持可读。

```
uv sync                 # 升级依赖
uv run pytest           # 回归验证
```

需要备份时，仅需复制工作目录下的运行数据目录与输出目录。运行数据目录内含 SQLite 数据库，其中保存工程登记信息、运行记录、告警复核状态、大模型端点配置与每次运行的源码快照；输出目录内含 SARIF 结果、HTML 报告与补丁文件。

卸载时直接删除项目目录与运行数据目录即可。系统不写注册表、不安装系统服务、不在系统目录留存文件。

4 命令行功能详解

本章按子命令逐一说明功能、主要参数与输出结果。所有子命令均支持 -h 查看完整参数列表。

4.1 scan 全量扫描

对目标工程执行一次完整检测，按 L1 构建复现、L3 双通道分析、L5 聚合输出的顺序运行，产出 SARIF 结果文件与 HTML 报告。

```
uv run ecdet scan examples/demo_rtos
uv run ecdet scan examples/demo_rtos --profile automotive
uv run ecdet scan examples/demo_rtos --no-build -o out
```

参数	说明
path	目标工程根目录
--profile	行业规则集预设，如 automotive、juliet
--no-build	跳过 L1 构建复现，直接扫描源码
--svd	指定 CMSIS-SVD 芯片描述文件，用于硬件语义建模
--changed	增量扫描，仅分析给定变更文件的影响集
--baseline	指定告警基线文件，输出新增/已解决/持续三态
--save-baseline	将本次结果保存为基线
-o / --out	输出目录，默认当前目录下的 out

4.2 init-ccdb 构建复现

仅执行 L1 构建复现，识别工程结构并生成编译数据库，便于在接入扫描前单独验证工程识别结果。

```
uv run ecdet init-ccdb path/to/project
```

4.3 rules 规则集查询

列出系统内置的全部检测规则，包括规则编号、标题、对应的 CWE 编号、所属标准与默认严重级别。

```
uv run ecdet rules
```

4.4 report 报告重建

以已有 SARIF 结果文件为输入重新生成 HTML 报告，便于在不重复扫描的前提下调整报告样式或补充说明。

```
uv run ecdet report results.sarif
```

4.5 triage 告警分诊

调用大模型对告警进行语义研判。分诊管线依次执行上下文裁剪、大模型首判、置信度不足时触发本地知识库检索增强重判、证据行号核对四步，结论回写 SARIF 与报告。

```
# 离线演示（确定性 Mock 大模型）
uv run ecdet triage out/results.sarif --project examples/svd_demo --mock -o out

# 接入本地大模型服务
export ECODET_LLM_BASE_URL=http://127.0.0.1:8000/v1
export ECODET_LLM_MODEL=Qwen2.5-Coder-7B-Instruct
uv run ecdet triage out/results.sarif --project examples/svd_demo -o out
```

分诊结果包含判定结论与置信度。系统设置三道幻觉抑制闸门：输出不可解析时降级为待复核；引用行号越界、指向注释行或未包含告警行时降级；大模型调用失败时降级。

4.6 fix 修复建议

对分诊判定为真缺陷的告警生成函数级补丁，并以三道工具验证把关：补丁应用后原告警必须消除且不引入新的 error 级告警；补丁须通过 gcc 语法检查；验证通过才产出统一 diff 补丁与修复记录。

```
uv run ecdet fix out/results.sarif --project examples/demo_rtos --mock -o out
# 产物：out/fixes/*.patch（统一 diff）+ out/fixes/fixes.json（采纳/拒绝与理由）
```

4.7 impact 影响集分析

给定变更文件，沿 include 反向闭包与调用图反向闭包计算影响集，用于大规模工程在每日集成中执行增量分析。

```
uv run ecdet impact examples/demo_rtos --changed src/sensor.c
uv run ecdet scan examples/demo_rtos --changed src/sensor.c
```

4.8 orchestrate 多智能体编排

以状态机形式顺序编排六类智能体：规划、静态通道、分诊并行扇出、动态通道、运行时监控裁决、修复验证闭环，最后经高危人工复核闸门进入 L5 聚合。

```
uv run ecdet orchestrate examples/demo_rtos --mock
```

4.9 serve 控制台服务

启动 Web 控制台 HTTP 服务。服务采用标准库 ThreadingHTTPServer 实现，扫描任务进入后台队列串行执行，页面以轮询方式获取任务进度。

```
uv run ecdet serve --port 8600
```

5 Web 控制台操作

Web 控制台以浏览器方式提供工程注册、扫描执行、代码浏览、告警处置、修复建议生成与大模型配置等全部操作，界面无需外网资源，离线可用。

5.1 界面结构

控制台采用左侧导航、右侧内容区的布局。左侧导航按“分析”“工程与代码”“结果与处置”三组组织为七个功能页，底部单列“大模型设置”入口。

分组	页面	功能说明
分析	总览	工程、运行、告警统计与最近运行列表
分析	扫描运行	扫描任务的状态、分级计数、构建通道与产物链接
工程与代码	工程管理	以本地路径、上传压缩包、Git 仓库三种方式注册工程并配置扫描参数
工程与代码	代码浏览	按文件浏览源码，标注告警行并提供文件内告警导航
工程与代码	版本对比	任选两次运行，比较文件变更与告警新增/已解决/持续
结果与处置	告警中心	告警筛选、逐条研判与状态流转
结果与处置	修复建议	批量生成补丁、展示 diff 与验证结论
—	大模型设置	配置大模型端点、测试连接

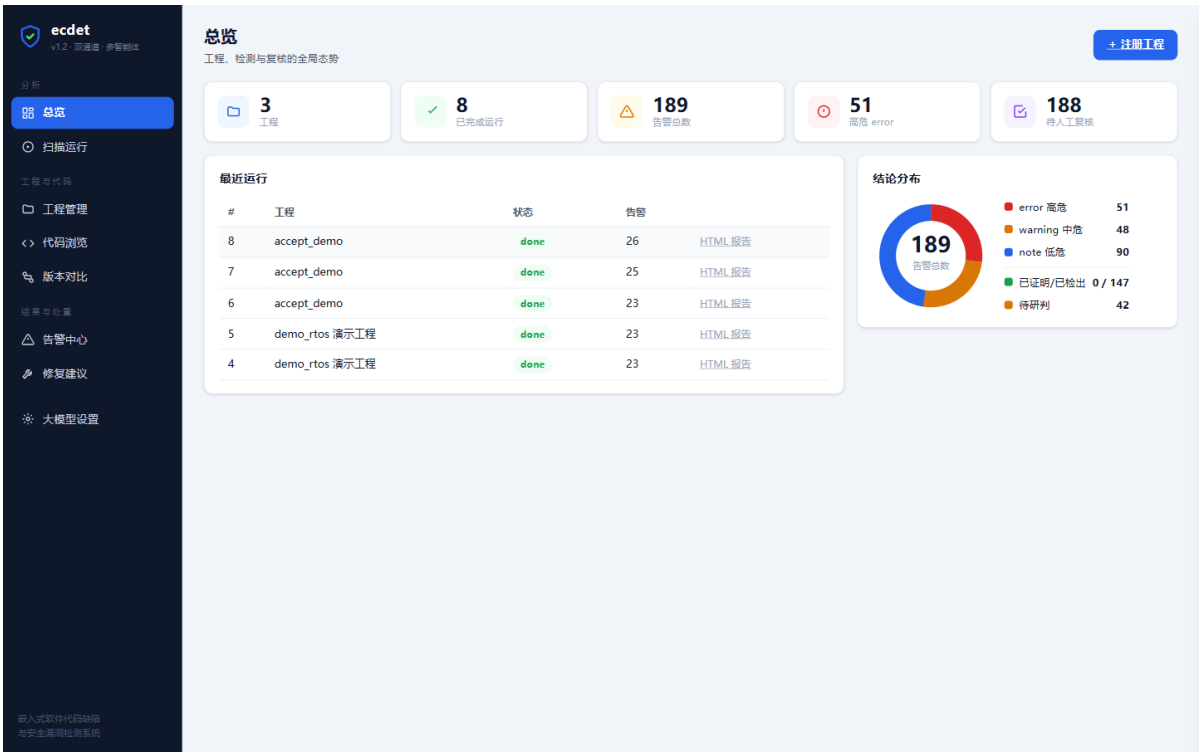


图 5-1　控制台总览页

5.2　总览

总览页集中呈现全局统计与最近运行记录。页面上方为五个统计卡片，分别显示工程总数、已完成运行数、告警总数、高危告警数与待人工复核数；中部左侧为最近运行列表，显示运行编号、所属工程、完成状态与告警数量，点击 HTML 报告链接可直接下载报告；右侧为结论分布环形图，按 error、warning、note 三级展示告警构成，并同时给出可证明性分布。

5.3　工程管理

工程管理页用于注册待检测工程。系统统一以“工程目录”作为输入单位，三种来源最终都归一为服务端可访问的目录。

来源	操作方式	归一处理
本地路径	填写服务端可访问的绝对路径	零拷贝直引，支持重复与增量扫描
上传压缩包	拖入 zip 或 tar.gz 文件	解压到工作区，解压过程校验路径逃逸
Git 仓库	填写仓库 URL 与分支，私有库可附访问令牌	浅克隆到工作区



图 5-2　工程管理页

注册工程时可一并设置扫描参数，包括行业规则集预设、芯片描述文件、大模型分诊模式（Mock 或真实端点）与动态通道开关。参数确定后点击开始扫描，任务提交至后台队列执行。

5.4　扫描运行

扫描运行页列出全部运行记录，每条记录展示运行编号、工程名称、执行状态、分级告警计数、构建通道与耗时，并提供 HTML 报告与 SARIF 结果文件的下载链接。进行中的任务自动轮询刷新，页面同时披露动态承载载体的可用性。

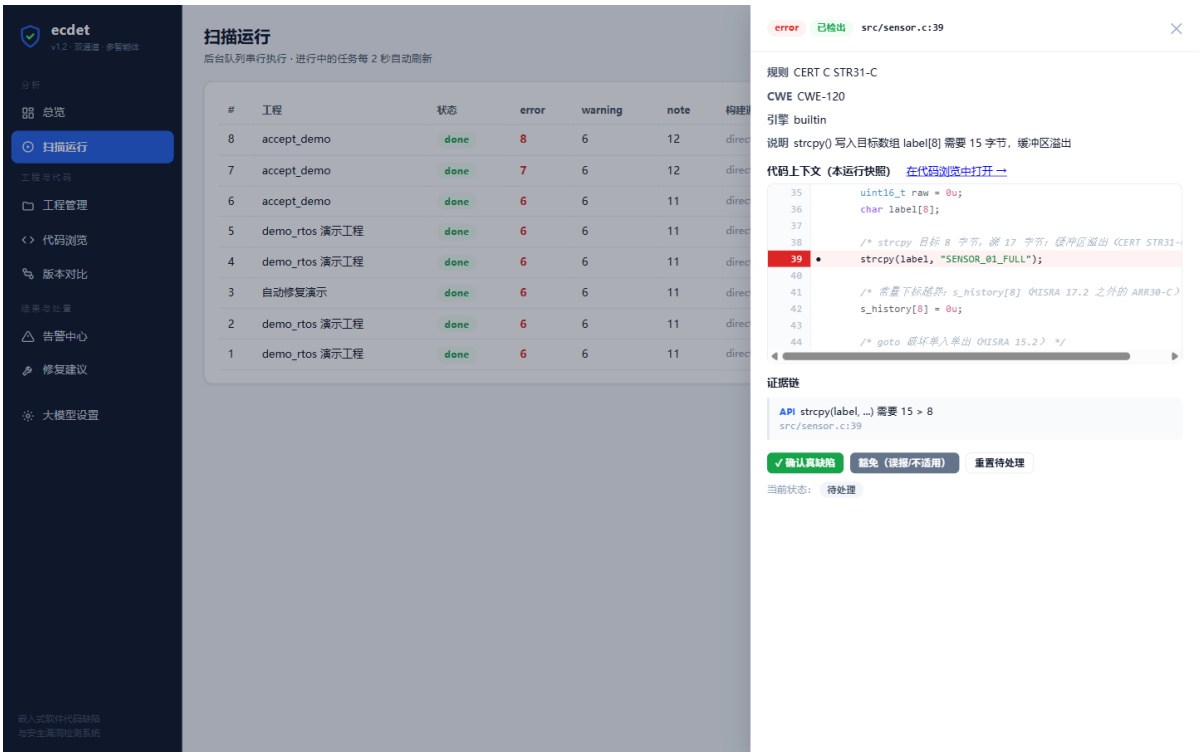


图 5-3　扫描运行页

5.5　代码浏览

代码浏览页以文件为中心组织信息。页面顶部选择工程与运行，左侧为文件树并按告警数量着色，右侧为整文件代码视图。

- 代码视图采用自研 C 语法高亮，告警所在行整行高亮，行号槽显示告警标记；
- 点击行号槽的告警标记可直接打开该告警的详情抽屉；
- 文件内告警可通过上下箭头逐条导航，标题栏显示当前告警在文件内的序号；
- 每次扫描完成时系统自动定格源码快照，修复改写源文件后仍可回溯历史视图。

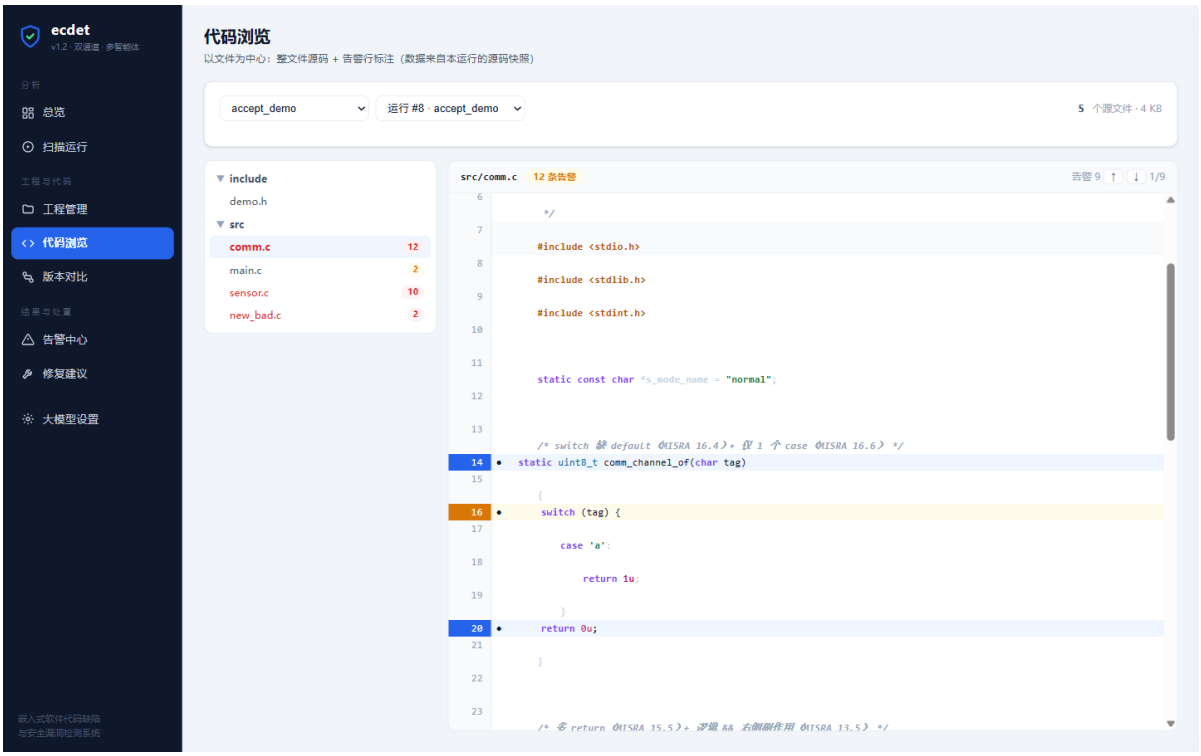


图 5-4　代码浏览页

5.6　版本对比

版本对比页用于比较同一工程任意两次运行的差异。选择基准运行与目标运行后，页面同时给出文件级变更与告警级差异两套结论。

- 文件级变更基于两次运行的快照清单哈希比对，分为新增、修改、删除三类；
- 告警级差异基于告警指纹比对，分为新增、已解决、持续存在三态，并给出各级别计数；
- 选择单个文件可查看统一 diff 或并排 diff，支持行内词级高亮，并在 diff 行边上标注该行对应的新增告警。

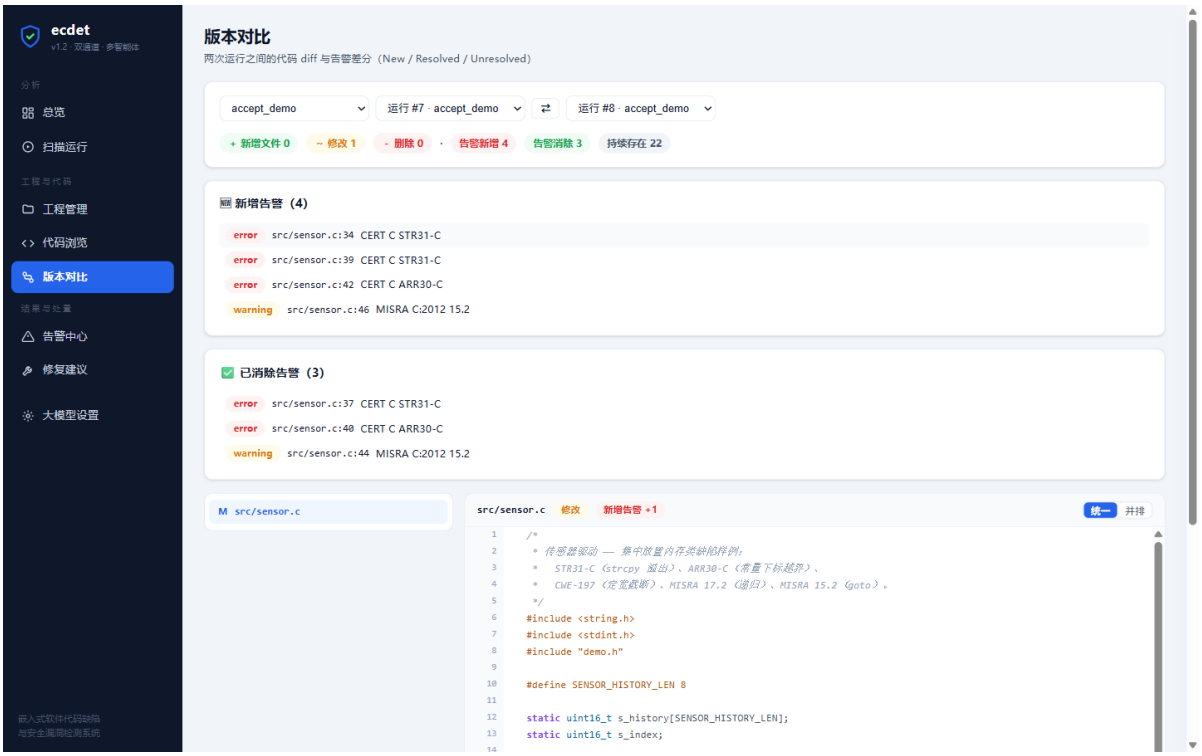


图 5-5　版本对比页

5.7　告警中心

告警中心以表格形式列出运行内的全部告警，支持按状态、严重级别与关键字筛选，表头提供严重级别计数芯片，可一键筛选。勾选告警后可直接进入修复流程。

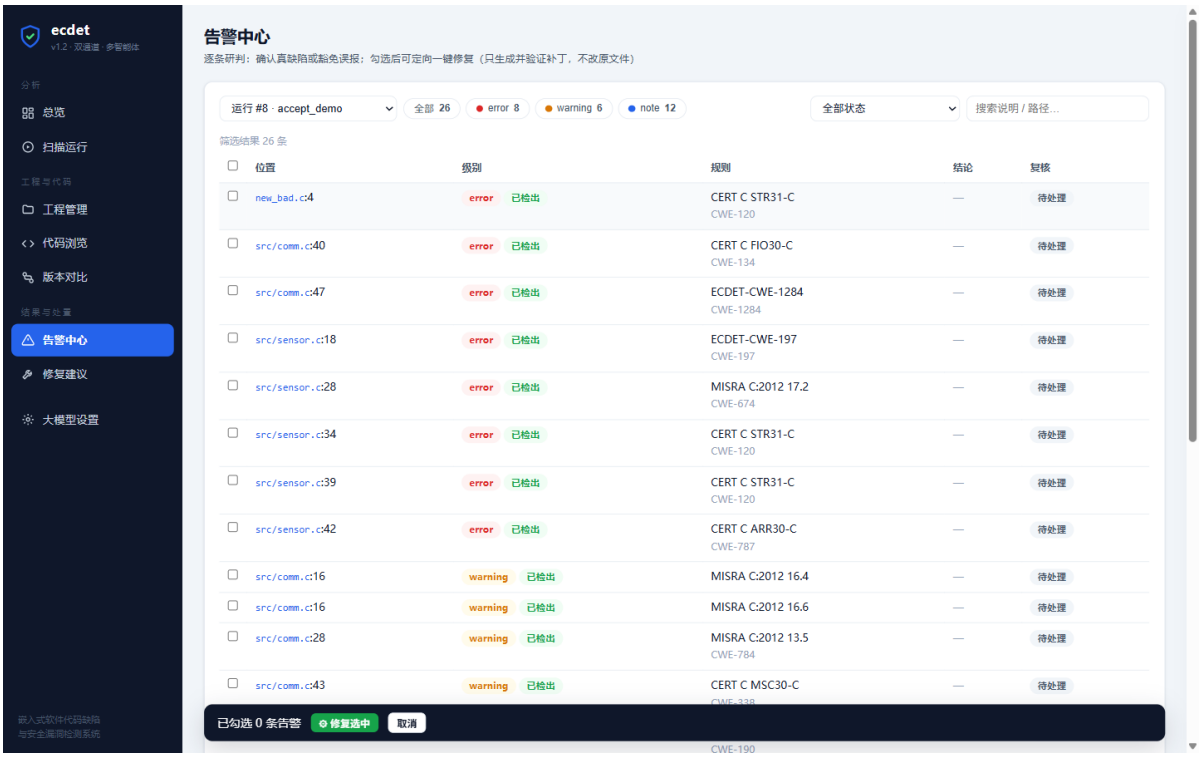


图 5-6　告警中心页

点击任意告警行可打开详情抽屉，抽屉内依次展示规则编号、CWE 编号、检测引擎、缺陷说明、代码上下文、证据链与处置按钮。代码上下文为本运行快照中告警行上下各五行，可一键跳转至代码浏览页对应位置；证据链列出支撑该告警的具体判据，例如接口调用与缓冲区长度比较结果。

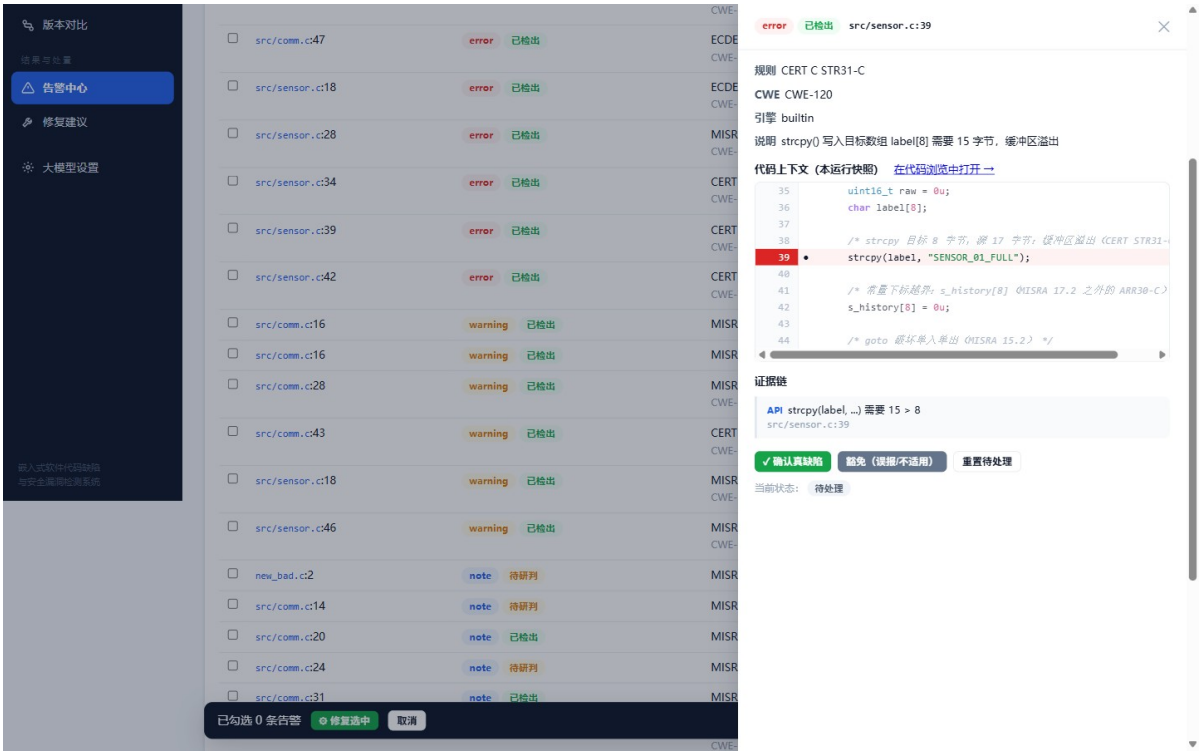


图 5-7　告警详情抽屉

处置按钮提供三种状态流转：确认真缺陷、豁免（误报或不适用）、重置待处理。状态变更即时写库，并影响后续修复流程中自动目标的选取范围。

5.8　修复建议

修复建议页对已完成运行批量生成补丁。页面顶部选择目标运行与工作模式，工作模式分为“仅生成建议（不改文件）”与“自动修复模式（写入源文件）”两种。

- 自动目标默认为高危且可修复的告警，包括人工确认真缺陷、大模型判真缺陷与动态通道已复现三类；也可在告警中心勾选后定向修复；
- 每个补丁展示采纳或拒绝结论、验证说明与统一 diff，采纳的补丁附带函数名与源码位置；
- 自动修复模式下，补丁写入前先备份源文件，写入后对全工程重扫，剩余告警进入下一轮迭代，直至收敛，并给出验收扫描统计。

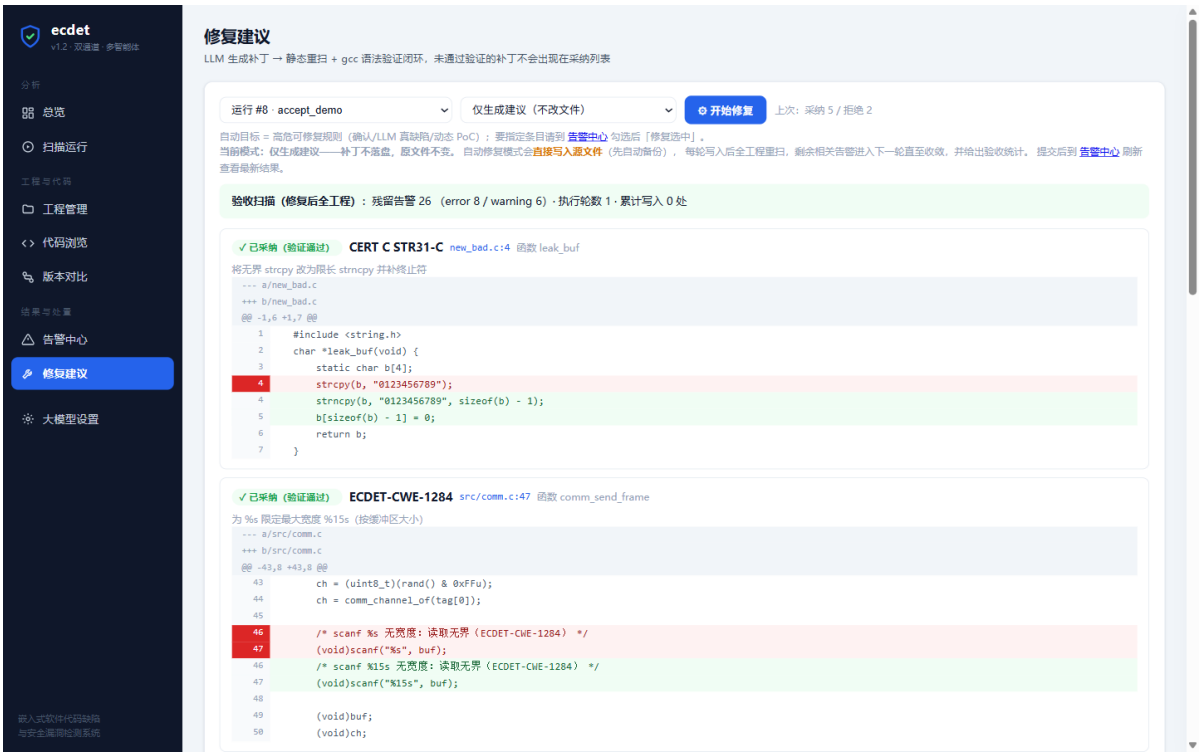


图 5-8 修复建议页

5.9 大模型设置

大模型设置页用于配置告警分诊与修复所使用的大模型端点。系统对接任意 OpenAI 兼容端点，可填入 Base URL、API Key 与模型名称，并通过测试连接按钮验证连通性。

- 凭据保存在本机数据库中，页面回显时对密钥做脱敏处理；
- 保存后分诊与修复立即使用真实端点；未配置时 Mock 模式保持离线可用；
- 数据不出域：可对接本地部署的推理服务，代码内容不发送至外部网络。

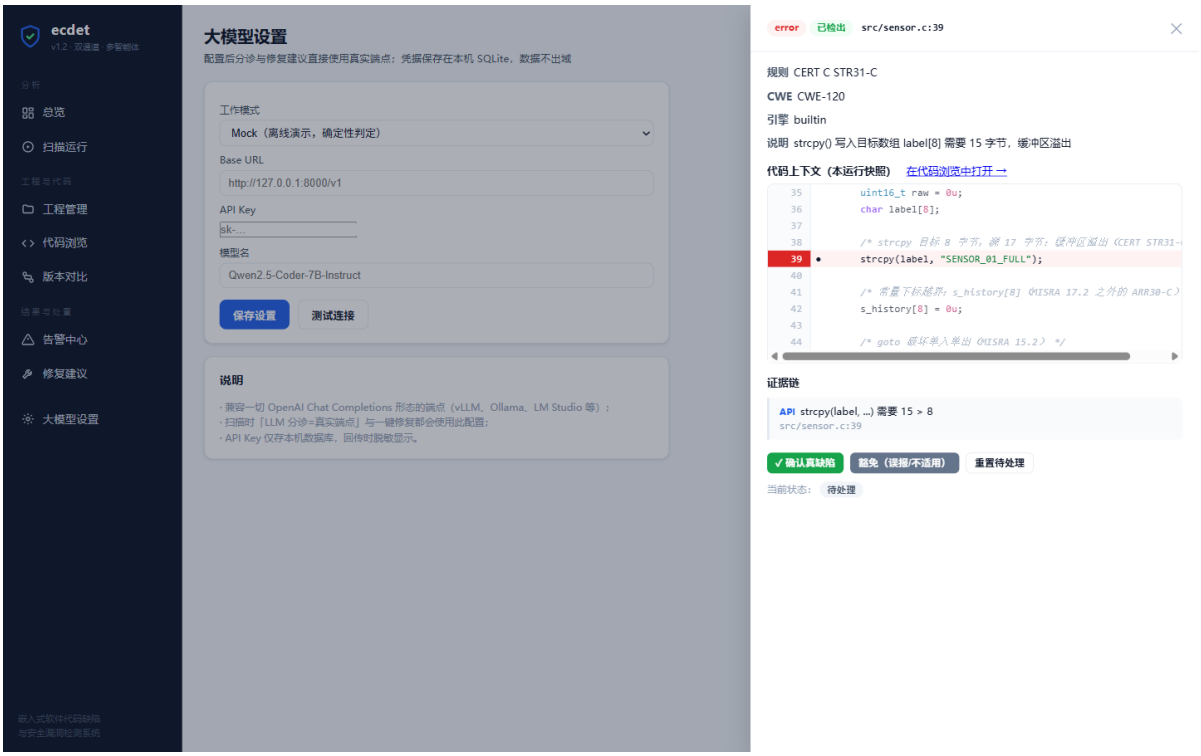


图 5-9　大模型设置页

5.10　界面通用交互

控制台各页遵循统一的交互约定，便于跨页操作时保持一致的操作手感。

交互元素	行为说明
原生下拉选择框	选择工程、运行、文件等对象，切换后当前页内容即时刷新
严重级别计数芯片	显示各严重级别的告警数量，点击即按该级别筛选，再次点击取消筛选
告警详情抽屉	从右侧滑出，点击右上角关闭按钮或按 Esc 键收起
整行点击	表格行可点击时鼠标显示为手型，点击即打开对应详情
右下角操作条	勾选告警后出现，显示已勾选数量并提供修复入口
吐司提示	操作完成或失败时在页面底部短暂提示，不阻塞后续操作

空态与加载态亦有统一处理：无数据时显示引导文案与跳转链接，数据加载过程中显示占位动效，避免出现空白页面引起误解。

5.11　状态与结论的含义

控制台中同时存在两类状态，二者相互独立，判读结果时需分别理解。

类别	取值	含义
运行状态	queued	任务已提交，等待队列调度
运行状态	running	正在执行扫描
运行状态	done	扫描完成，结果可供浏览与处置
运行状态	failed	扫描失败，失败原因记录在运行详情中
人工复核状态	待处理	尚未人工研判
人工复核状态	已确认	人工判定为真实缺陷，进入修复自动目标
人工复核状态	已豁免	判定为误报或不适用，不计入修复范围

6 检测能力说明

6.1 内置规则集

系统内置 30 条检测规则，分为四类，全部规则在关闭外部引擎时仍可独立运行。

类别	数量	规则编号与内容
MISRA C:2012 子集	17	2.5、7.2、8.7、8.9、10.3、10.4、13.5、14.2、15.2、15.5、16.4、16.6、17.2、17.3、21.1、21.3、21.6
CERT C 子集	5	PRE31-C、MSC30-C、STR31-C、ARR30-C、FIO30-C
缺陷规则	4	CWE-242（禁用 gets）、CWE-197（字面量截断）、CWE-1284（scanf 无界）、CWE-563（未使用参数）
硬件语义规则	4	ECDET-MMIO-ACCESS（寄存器读写属性违例）、ECDET-ISR-RACE（中断与任务竞态）、ECDET-VOLATILE（共享变量缺 volatile）、ECDET-ISR-BLOCK（中断服务程序内调用阻塞接口）

6.2 外部引擎接入

系统在启动时探测 cppcheck、clang-tidy 与 ESBMC 是否可用，可用即接入并纳入统一告警模型，不可用则跳过并在运行日志中记录。外部引擎输出经统一转换后与内置规则告警合并去重，在结果中通过引擎字段区分来源。

6.3 可证明性分级

为解决静态分析误报难以甄别的问题，系统为每条告警标注可证明性等级。

等级	含义	判定依据
Proven	已证明	抽象解释的常量传播证明数组下标越界，或有界模型检验的界内验证失败
Detected	已检出	确定性词法或规则证据核对确认，或动态执行已复现
Suspected	疑似	静态可疑但缺少确定性证据，需大模型或人工研判

当抽象解释证明下标在界内时，系统不产生告警，从而抑制此类误报。

6.4 合规标准映射

系统为每条内置规则建立 CWE 编号与 GJB 8114-2013 主题级映射，报告中的合规记录表同时呈现 MISRA C、CERT C、内部规则编号与 GJB 双口径结论，便于开展编码规范符合性检查与整改证据留存。

6.5 动态分析通道

动态通道与静态通道并列，独立产出可复现的缺陷证据。其流程包含四个环节：

- 激励建模：以确定性特殊值表（零值、边界值、回绕点）与乘法散列采样生成入参组合；若提供芯片描述文件，则按寄存器语义收窄取值域，实现一次建模、静动两用；
- harness 生成与编译修正：以解析得到的参数类型为骨架生成测试驱动，激励组合表嵌入 C 数组；链接期未解析符号与编译器隐式声明错误自动打桩并重新编译；
- 执行载体：按吞吐与保真度分为宿主原生执行、CPU 级仿真、外设级仿真三档，运行时披露各档可用性；
- 结果判定：以崩溃退出码分类、未定义行为检测器输出与看门狗超时作为判据，命中即生成可一键重放的概念验证样本，并随告警证据写入结果文件。

6.6 多智能体协同

系统将告警处置流程分解为六类智能体，通过结构化状态对象逐级交接。大模型负责语义推理与流程编排，不承担最终裁决；所有结论须经证据核对与工具验证。成本控制上采用分层路由，低级别告警不进入大模型环节。系统同时提供单智能体串行形态，用于能力对比与降级运行。

6.7 误报抑制机制

静态分析工具在嵌入式工程上往往产生大量误报，若不加以抑制，会使整改工作失去焦点。系统在执行链路上设置了四道抑制环节，逐级收敛。

环节	作用	效果
抽象解释证明	对数组下标进行常量传播与区间推导；证明在界内时不产生告警	消除上下界可由常量推导的越界误报
硬件语义上下文	按芯片描述文件判定寄存器读写权限；按中断上下文判定并发关系	消除对只读寄存器写入等不成立的告警

环节	作用	效果
证据行核对	大模型引用的证据行必须位于被测函数区间内、为真实代码行、且包含告警行本身，否则降级为待复核	拦截引用错误位置得出的错误结论
工具验证闭环	修复补丁须通过静态重扫与编译器语法检查，未通过不予采纳	阻止引入新问题的补丁流入代码库

四道环节并不改变原始告警的检出情况，而是逐级提升剩余告警的可信度：通过前两道环节的告警标记为已证明或已检出，通过第三道环节的结论附带可信度评分，通过第四道环节的补丁才被视为可用修复。

7 输出产物

7.1 SARIF 结果文件

扫描结果统一输出为 SARIF 2.1.0 格式的 results.sarif 文件。该格式为业界通用的静态分析结果交换标准，可被主流代码扫描平台与缺陷管理系统直接消费。文件内容包括规则定义、告警位置、严重级别、证据链、可证明性等级与大模型研判结论。

7.2 HTML 检测报告

每次扫描同步生成 report.html 单文件报告，内嵌样式与脚本，可离线打开与归档。报告包含检测概览、分级统计、可证明性分布、逐条告警明细与合规记录表。

7.3 补丁与修复记录

修复流程的产物位于输出目录的 fixes 子目录：每个采纳的补丁保存为独立 patch 文件，内容为统一 diff 格式，可直接由版本控制工具应用；修复记录文件 fixes.json 保存每条告警的采纳或拒绝结论、验证说明与关联的告警编号。

7.4 基线文件

执行扫描时指定保存基线参数，系统将本次告警集合以指纹形式写入基线文件。后续扫描可加载该基线，输出新增、已解决与持续存在三态差异，使增量检查聚焦于本次变更引入的问题。

8 典型使用流程

8.1 首次全量扫描

首次接入一个工程时，建议先在命令行完成一次全量扫描，确认工程识别与规则命中情况。

```
uv run ecdet init-ccdb path/to/project
uv run ecdet scan path/to/project --profile automotive -o out
```

扫描完成后检查输出目录下的 results.sarif 与 report.html。若工程识别失败，可使用跳过构建的参数直接扫描源码目录。

8.2 大模型分诊

在全量扫描结果基础上执行分诊，过滤误报并标注置信度。首次使用可先以 Mock 模式验证流程，再切换至本地推理服务。

```
uv run ecdet triage out/results.sarif --project path/to/project --mock -o out
uv run ecdet triage out/results.sarif --project path/to/project -o out
```

8.3 一键修复与验收

在控制台打开修复建议页，选择目标运行与工作模式，点击开始修复。系统对可修复告警生成补丁并逐条验证，采纳的补丁展示 diff，拒绝的补丁给出原因。若采用自动修复模式，补丁写入源文件后系统自动重扫并迭代，直至剩余告警不再变化，最后给出验收扫描统计。

8.4 增量扫描与版本对比

日常集成中，建议以变更文件为输入执行增量扫描，并以基线差分聚焦新增告警。

```
uv run ecdet scan path/to/project --save-baseline -o out
uv run ecdet scan path/to/project --baseline out/baseline.json -o out
uv run ecdet impact path/to/project --changed src/sensor.c
```

在控制台的版本对比页，可直接选择两次运行，查看文件级变更与告警三态差异，并逐个文件查看 diff。需要说明的是，告警指纹包含行号，代码整体移动会使告警计为新增与已解决的组合，此时建议结合文件 diff 判断实际影响。

9 常见问题与使用限制

9.1 常见问题

现象	原因与处理
扫描结果中无外部引擎告警	cppcheck、clang-tidy 或 ESBMC 未安装或不在 PATH 中，系统已自动跳过。安装后重新扫描即可接入。
分诊结果全部为待复核	大模型端点不可达或返回内容无法解析为结构化结果，系统按幻觉抑制规则降级。请在设置页测试连接后重试。
修复建议为空	没有满足自动目标的告警。可在告警中心勾选告警后定向修复，或确认是否存在可修复的高危告警。
动态通道未产出证据	宿主未安装 gcc 或被测函数的依赖无法打桩。检查运行日志中披露的载体可用性。
版本对比提示同一运行	基准运行与目标运行不能相同，请选择两次不同的运行。
端口被占用导致启动失败	使用端口参数指定其他端口，例如指定 8600 之外的端口。

9.2 使用限制

- 内置规则覆盖 MISRA C:2012 与 CERT C 的子集，非全量覆盖；完整合规检查建议与外部引擎结果合并使用。
- GJB 8114-2013 映射为主题级映射，规则编号需依据授权文本核实，报告中对未核实项予以标注。
- 动态通道以宿主原生执行为主，CPU 级与外设级仿真载体为扩展项；宿主执行不模拟外设行为，涉及寄存器与外设交互的缺陷需结合仿真载体确认。
- 告警指纹包含行号，代码整体移动会导致告警计为新增与已解决，增量结果需结合文件 diff 判读。
- 大模型结论为辅助研判，不作为最终裁决依据；高危告警仍建议经人工复核确认。

10 数据与安全管理

嵌入式项目的源码属于高敏感资产，检测系统在部署与运行上采取“数据不出域”的基本策略，并提供可追溯的运行记录。

10.1 部署边界

系统全部组件均可在内网或单机部署，运行过程不主动向外部网络发起请求。大模型分诊为可选能力，可对接本地部署的推理服务（vLLM、Ollama、LM Studio 等），源码上下文仅在本地网络内传输。未配置大模型时，系统以 Mock 模式运行，分诊流程保持完整但结论为确定性模拟结果，适用于离线演示与流程验证。

10.2 凭据管理

大模型端点的访问密钥保存在本机数据库的配置表中，页面回显时仅显示脱敏结果，完整密钥不回传前端。提交配置时若密钥字段留空，系统保留原有密钥不作覆盖，避免因脱敏显示导致密钥被误清空。

10.3 运行数据留存

系统在运行数据目录中保存三类数据：工程登记信息（名称、来源类型、工作区路径）、扫描运行记录（参数、状态、产物路径、分级计数）与告警复核状态（逐条告警的处理结论）。每次扫描完成时另存该次运行的源码快照与哈希清单，用于代码浏览与版本对比，使源码在后续被修复改写后仍能回溯当时的视图。

10.4 路径安全

工程来源中的上传压缩包在解压时校验解压目标是否越出工作区，防止路径逃逸；源码浏览与文件差异接口均以运行快照清单为白名单，仅允许访问清单内登记的文件，不接受任意路径读取请求。

10.5 可追溯性

每条告警记录规则编号、检测引擎、文件路径与行号、证据内容与可证明性等级；每次修复记录对应的告警编号、采纳或拒绝结论与验证说明；人工复核的状态变更即时落库。上述记录共同构成可追溯的处置链条，便于事后复核与合规检查。

11 附录

附录 A 内置规则清单

下表列出系统内置的 30 条检测规则。严重级别按 MISRA 与 CERT 原始分级标注，Mandatory 与 Required 对应高优先级，Advisory 对应提示级。

规则编号	级别	判定方式	规则内容
MISRA C:2012 2.5	Advisory	语法	宏已定义但在翻译单元中未被使用
MISRA C:2012 7.2	Required	语法	无符号类型的整型常量表达式应带 u/U 后缀
MISRA C:2012 8.7	Required	工程	仅被一个翻译单元引用的函数应具有内部链接
MISRA C:2012 8.9	Advisory	工程	仅被一个翻译单元引用的全局变量应具有内部链接
MISRA C:2012 10.3	Required	类型	赋值表达式的值不应发生基本类型类别降级
MISRA C:2012 10.4	Required	类型	常规算术转换的双操作数类型类别应一致
MISRA C:2012 13.5	Required	语法	逻辑运算右操作数不应包含持续副作用
MISRA C:2012 14.2	Required	语法	for 循环计数变量不应在循环体内被修改
MISRA C:2012 15.2	Required	语法	不应使用 goto 语句
MISRA C:2012 15.5	Advisory	语法	函数末尾应有单一退出点
MISRA C:2012 16.4	Required	语法	每个 switch 语句应具有 default 标签
MISRA C:2012 16.6	Required	语法	每个 switch 语句应至少包含两个 case 标签
MISRA C:2012 17.2	Required	语法	不应使用递归
MISRA C:2012 17.3	Mandatory	工程	不应隐式声明函数
MISRA C:2012 21.1	Required	语法	不应定义或使用保留标识符
MISRA C:2012 21.3	Required	语法	不得使用标准库的内存分配与转换函数
MISRA C:2012 21.6	Required	语法	不得使用标准输入输出库函数
CERT C PRE31-C	Required	语法	宏参数在宏体中每次出现都应加括号
CERT C MSC30-C	Required	语法	不得使用 rand() 生成安全相关随机数

规则编号	级别	判定方式	规则内容
CERT C STR31-C	Required	语法	字符串与内存拷贝须保证目标空间充足
CERT C ARR30-C	Required	语法	不得以常量下标访问数组越界位置
CERT C FIO30-C	Required	语法	格式化 I/O 的格式串必须是字面量
ECDET-CWE-242	Mandatory	语法	使用 gets()，存在必然的缓冲区溢出风险
ECDET-CWE-197	Mandatory	语法	整型字面量初始化超出定宽类型表示范围
ECDET-CWE-1284	Required	语法	scanf 家族使用 %s 且未限定宽度
ECDET-CWE-563	Advisory	语法	函数参数在函数体内未被使用
ECDET-MMIO-ACCESS	Mandatory	数据流	违背芯片描述文件定义的寄存器访问权限
ECDET-ISR-RACE	Required	数据流	中断与任务共享全局变量且访问侧无临界区保护
ECDET-VOLATILE	Advisory	语法	中断共享变量未声明 volatile
ECDET-ISR-BLOCK	Mandatory	语法	中断服务程序中调用阻塞或非可重入接口

附录 B 检测结果文件示例

以下为结果文件的结构示例，展示规则定义与单条告警的字段组织方式。实际文件中字段完整，此处仅截取关键部分。


```
{
  "version": "2.1.0",
  "$schema": "https://json.schemastore.org/sarif-2.1.0.json",
  "runs": [
    {
      "tool": {
        "driver": {
          "name": "ecdet",
          "version": "1.2.0",
          "rules": [
            {
              "id": "CERT-C-STR31-C",
              "name": "Str31C",
              "shortDescription": {
                "text": "字符串与内存拷贝须保证目标空间充足"
              },
              "properties": {
                "cwe": "CWE-120",
                "standard": "CERT C",
```

```
              "gjb": "GJB 8114-2013 相关主题"
            }
          }
        ]
      }
    },
    "results": [
      {
        "ruleId": "CERT-C-STR31-C",
        "level": "error",
        "message": {
          "text": "strcpy 目标 8 字节，源 17 字节，缓冲区溢出"
        },
        "locations": [
          {
            "physicalLocation": {
              "artifactLocation": { "uri": "src/sensor.c" },
              "region": { "startLine": 39, "startColumn": 5 }
            }
          }
        ],
        "properties": {
          "engine": "builtin",
          "provability": "Detected",
          "evidence": [
            "API strcpy(label, ...) 需要 15 > 8"
          ],
          "triage": {
            "verdict": "TRUE_POSITIVE",
            "confidence": 0.92,
            "rag_used": false
          }
        }
      }
    ]
  }
]
}
```

字段说明：level 取 error、warning、note 三级；provability 取 Proven、Detected、Suspected 三级；triage 块仅在执行分诊后出现，包含判定结论、置信度与是否触发知识检索。

附录 C　控制台服务接口一览

控制台前端通过下列接口与服务端交互，接口同时可被自动化脚本直接调用。

方法	路径	用途
GET	/api/summary	全局统计：工程数、运行数、告警数与分级计数
GET	/api/projects	工程列表
POST	/api/projects	注册工程（本地路径、Git 仓库或压缩包上传）
GET	/api/runs	运行列表，可按工程过滤

方法	路径	用途
POST	/api/runs	提交扫描任务，参数含规则集预设、芯片描述、分诊模式与动态开关
GET	/api/runs/{id}	单次运行详情，含分级计数
GET	/api/runs/{id}/alerts	该运行的告警列表，支持按级别、规则、引擎、状态与关键字筛选
GET	/api/runs/{id}/report	下载 HTML 报告
GET	/api/runs/{id}/sarif	下载 SARIF 结果文件
GET	/api/runs/{id}/files	该运行的源码快照文件树与逐文件告警计数
GET	/api/runs/{id}/file	读取快照内单个文件内容，用于代码浏览
GET	/api/runs/{id}/diff	两次运行的文件级变更与告警三态差异
GET	/api/runs/{id}/diff/file	单文件差异内容，支持统一或并排两种形式
GET	/api/runs/{id}/fixes	该运行的修复任务与补丁列表
POST	/api/runs/{id}/fix	提交修复任务，参数含定向告警列表与工作模式
GET	/api/alerts/{id}	单条告警详情，含证据链与代码上下文
POST	/api/alerts/{id}/status	更新人工复核状态
GET	/api/settings	读取大模型配置，密钥脱敏
POST	/api/settings	保存大模型配置
POST	/api/settings/test	测试大模型端点连通性
GET	/api/carriers	查询动态执行载体可用性

附录 D　输出目录结构

一次扫描在输出目录下产生下列文件与子目录。

```
out/
├── results.sarif          统一结果文件（SARIF 2.1.0）
├── report.html            单文件检测报告
├── baseline.json          告警基线（启用保存基线时生成）
└── fixes/
    ├── *.patch            采纳补丁的统一 diff 文件
    └── fixes.json         修复记录：采纳/拒绝结论与理由

webui_data/                控制台运行数据（使用控制台时生成）
├── ecdet_web.db           SQLite 数据库
```

```
├── projects/        工程工作区
├── runs/            逐次运行的输出与记录
├── snapshots/       逐次运行的源码快照与哈希清单
└── backups/         自动修复模式下的源文件备份
```

附录 E　术语表

术语	说明
SARIF	静态分析结果交换格式，本系统采用 2.1.0 版本作为唯一结果模型
MISRA C	汽车工业软件可靠性协会发布的 C 语言编码规范
CERT C	美国卡内基梅隆大学软件工程研究所发布的 C 语言安全编码标准
CWE	通用缺陷枚举，对软件缺陷类型的标准化编号
GJB 8114	中国国家军用标准《C/C++语言编程安全子集》
SVD	CMSIS-SVD，ARM 定义的系统视图描述文件，用于描述芯片寄存器属性
ISR	中断服务程序
MMIO	内存映射输入输出
RTOS	实时操作系统
harness	测试驱动，为执行被测函数而自动生成的调用框架
PoC	可复现缺陷的最小验证样本
抽象解释	以抽象域上的不动点计算推导程序性质的静态分析技术
有界模型检验	将程序性质验证归约为可满足性求解的自动化验证技术
可证明性分级	本系统对告警证据强度划分的已证明、已检出、疑似三级
基线	某次扫描的告警集合快照，用于后续扫描的差异比对
增量分析	仅分析变更文件影响集的扫描方式
告警指纹	由规则编号、文件路径、行号与上下文哈希构成的告警唯一标识